JavaScript Playground

A beginner's complete guide to the pasted code

This guide explains the whole script from top to bottom: every numbered source line, the important words and symbols, how the browser page connects to the JavaScript, and what happens when someone types, clicks Run, enters a prompt answer, or waits for a timer.

The explanation assumes no prior JavaScript knowledge. It uses small examples and keeps the original program's purpose in view: a browser-based code editor with highlighting, output, a prompt dialog, and teaching notes.

A useful mental picture

The script is a coordinator. It connects page elements, responds to events, formats code and output, runs the learner's code, and then builds explanations. The browser supplies the page, event system, timers, and storage; the script tells those parts what to do.

Contents

- 1. The page and the JavaScript: how they connect
- 2. Beginner vocabulary and symbols used throughout
- 3. The source explained line by line (lines 1-78)
- 4. The full Run journey, traced with examples
- 5. Async functions, Promises, await, and prompt dialogs
- 6. setTimeout and the click ripple
- 7. Regular expressions and the teaching-rule engine
- 8. A glossary of important variables and browser methods
- 9. What the program does not do, plus careful-reading notes
- Appendix. The complete pasted source, numbered

The source has 78 physical lines. Blank lines are explained too: they improve readability but do not execute.

1. The page and the JavaScript: how they connect

A web page is commonly made from three pieces: HTML describes elements, CSS controls how they look, and JavaScript makes them respond. This pasted file is the JavaScript piece. It expects the HTML to already contain the editor, output panel, prompt dialog, buttons, and other elements it selects.

For example, an HTML element such as `textarea id="code"` can be found by `document.querySelector('#code')`. The JavaScript then stores that element in a variable and attaches event handlers to it.

High-level connection map

PAGE LOADS	USER TYPES	USER RUNS	CODE FINISHES
Restore saved text/theme; sync view	Highlight + save text	Compile and execute; capture output/errors	Set status; explain source lines

The code uses IDs and classes such as #code, #run, .card, and .btn. An ID should identify one element; a class can be shared by many elements. If the expected HTML element is missing, a selector may return null, and trying to use a property such as .value on null can cause an error.

2. Beginner vocabulary and symbols

Term	Meaning in this program
Value	A piece of information, such as the text "hi", the number 3, or true.
Variable	A named place that lets the program refer to a value. Examples here include code, src, and err.
const	Declares a name that cannot be assigned a different value later. If the value is an object, its properties may still be changed.
let	Declares a name that can be assigned a new value later. This code uses it for changing state such as the current error.
Function	A reusable set of instructions. It can receive inputs called parameters and can return a result.
Arrow function =>	A shorter way to write many functions. In s => s.length, s is the input and the expression after => is the result.
Callback	A function passed to another operation so it can be called later. Event handlers, array forEach, and setTimeout use callbacks.
Object	A collection of named properties. For example, con has properties such as log, warn, and error.
Array []	An ordered list. R is an array of teaching rules; each rule is itself an array with a pattern, tag, and explanation function.
Set	A collection that stores unique values. tags uses a Set so a topic like Variables is shown only once.
String	Text. Strings here use single quotes, double quotes, or backticks.
Template literal `...`	A string written with backticks. The ${...} parts are replaced with values, so `Hi ${name}` can include a variable.
Dot .	Accesses a property or method, such as code.value or out.append(...).
Parentheses ()	Call a function, group an expression, or hold function parameters.
Braces {}	Mark a block of instructions or an object literal, depending on context.
Comma ,	Separates arguments, properties, or declarations. Several declarations on one pasted line are separated by commas.
Semicolon ;	Often marks the end of a statement. JavaScript can insert some automatically, but explicit semicolons make boundaries visible.
if / else	Chooses which instructions to run based on whether a condition is true.
try / catch	Runs code that may fail; if an error is thrown, catch receives it so the program can respond.
return	Sends a value back from a function and ends that function call.
null	An intentional no-value result. Here it represents cancelling a prompt.
undefined	A value meaning no value was provided or found.
Boolean	A true-or-false value.
event	An object the browser provides when something happens, such as a key press, click, or pointer movement.

Common operators

- **=** - Assign a value: code.value = text stores text in the editor element.
- **===** - Strict equality: compares values without quietly converting their types.
- **!** - Logical NOT: !condition means the condition is false.
- **&&** - Logical AND: both sides must be true.
- **||** - Logical OR: the left side is used if truthy; otherwise the right side is used. load('cf') || code.value uses the saved value if it is nonempty.
- **? :** - Ternary operator: condition ? valueIfTrue : valueIfFalse. It is a compact if/else expression.
- **...** - Spread/rest syntax. In (...a), it collects any extra arguments into the array a.
- **++** - Adds one to a number. i + 1 is used for one-based line numbering.
- **+** - Adds numbers, joins strings, or both depending on the values. 'l ' + t joins text.
- **&& in conditionals** - Runs the next expression only when the left side is true. The code uses this to guard actions.

3. The source explained line by line

Line numbers below refer to the 78 physical lines in the pasted file. Each source line is shown in a monospaced box; long lines wrap for the page. The explanation follows the code from top to bottom.

A. Element references and persistence

Line 001 - Find the page elements

```
001 | const $=s=>document.querySelector(s),code=$('#code'),hl=$('#hl'),ln=$('#ln'),out=$('#out'),why
    | =$('#why'),stat=$('#stat');
```

The first declaration creates $ as a selector shortcut. The comma-separated declarations immediately use it to find six elements by ID. code, hl, ln, out, why, and stat will be reused throughout the program. The page must define these IDs before this script runs.

Line 002 - Save and load browser data

```
002 | const save=(k,v)=>{try{localStorage[k]=v}catch{}},load=k=>{try{return
    | localStorage[k]}catch{}};
```

save(k,v) tries to set localStorage[k] to v. load(k) tries to read it. Both catch storage errors and intentionally do nothing on failure, so private browsing or browser policy does not crash the whole script. The empty catch blocks are legal JavaScript.

Line 003 - Blank line

No instruction runs here. The blank line separates setup from the syntax-highlighting section.

B. Syntax highlighting

Line 004 - Section comment

```
004 | /* ---------- Syntax highlighting: split code into tokens, wrap each in a coloured span
    | ---------- */
```

Everything between /* and */ is a comment. The browser ignores it; the author uses it as a heading for humans.

Line 005 - Keywords collection

```
005 | const KW=new Set('const let var function return if else for while do switch case break
    | continue new class extends try catch finally throw async await of in typeof instanceof this
    | null undefined true false default yield delete void'.split(' '));
```

new Set(...) creates a collection of unique strings. The long text is split at spaces into words like const, while, and return. Later, KW.has(word) checks whether a highlighted token is one of these language keywords.

Line 006 - Built-in names collection

```
006 | const BI=new Set('console Math JSON Object Array Number String Boolean Promise Date Map Set
    | document window'.split(' '));
```

BI contains common JavaScript and browser names such as console, Math, Promise, document, and window. The highlighter colors these names separately from ordinary variable names.

Line 007 - Escape text before HTML display

```
007 | const esc=s=>s.replace(/&/g,'&amp;').replace(/</g,'&lt;');
```

esc(s) replaces ampersand and less-than characters with HTML-safe entities. This allows typed code to be placed inside innerHTML as visible text rather than accidentally becoming markup.

Line 008 - Regex-group comment

```
008 | // groups: 1 comment, 2 string, 3 number, 4 name before "(", 5 other name, 6 operator
```

This is a single-line comment explaining the six capture groups used by TOK. A capture group is a parenthesized part of a regular expression whose match can be read as m[1], m[2], and so on.

Line 009 - Token-finding regular expression

```
009 | const TOK=/(\/\/.*|\/\*[\s\S]*?\*\/)|("(?:\\.|[^"\\\n])*"|'(?:\\.|[^'\\\n])*'|`(?:\\.|[^`\\])*
    | `)|(\b\d+\.?\d*\b)|([A-Za-z_$][\w$]*)(?=\s*\()|([A-Za-z_$][\w$]*)|([=+\-*/%<>!&|?:]+)/g;
```

TOK describes patterns the highlighter recognizes. The alternatives separated by | look for comments, quoted strings, numbers, a name before an opening parenthesis, other names, and operators. [\s\S] means any character including a newline; *? means repeat as little as possible; \d means digit; \w means a letter, digit, or underscore; and g means scan all matches. This is a lightweight colorizer, not a full JavaScript parser.

Line 010 - Turn tokens into colored HTML

```
010 | const hi=s=>{let r='',i=0,m;TOK.lastIndex=0;while(m=TOK.exec(s)){const t=m[0],c=m[1]?'com':m[2
    | ]?'str':m[3]?'num':m[4]||m[5]?(KW.has(t)?'kw':BI.has(t)?'bi':m[4]?'fn':''):'op';r+=esc(s.slice
    | (i,m.index))+(c?`<span class="${c}">${esc(t)}</span>`:esc(t));i=TOK.lastIndex}return
    | r+esc(s.slice(i))};
```

hi(s) scans a string s. It initializes output r and cursor i, resets TOK, then repeatedly calls TOK.exec(s). For every match, t is the token and c is the CSS class chosen from the matching group. KW.has and BI.has classify names. s.slice(i,m.index) preserves text before the token; esc(t) protects token text; the template literal creates a span with class c. i advances to TOK.lastIndex. At the end, the unprocessed tail is escaped and returned.

Line 011 - Blank line

No code runs. It separates the highlighter from the editor synchronization section.

C. Editor synchronization and keyboard behavior

Line 012 - Editor-layout comment

```
012 | // The coloured <pre> sits under a transparent <textarea>; keep text, line numbers and scroll
    | in sync
```

This comment describes the HTML/CSS design: a colored preformatted element is visually underneath a transparent textarea. The user types into the textarea; the colored layer shows the same characters with color. CSS is needed for the layers to line up.

Line 013 - Refresh highlighted view and line numbers

```
013 | const sync=()=>{hl.innerHTML=hi(code.value)+'\n
    | ';ln.textContent=code.value.split('\n').map((_,i)=>i+1).join('\n')+'\n '};
```

sync reads code.value. It sends that source to hi and places the resulting HTML in hl. It also splits the source at newline characters, converts each line index i into i+1, and joins the numbers with newlines for ln. The final extra newline and space help scrolling stay aligned.

Line 014 - React when text changes

```
014 | code.oninput=()=>{sync();save('cf',code.value)};
```

oninput is an event handler: the browser calls it after editing. It refreshes highlighting/line numbers, then saves the current text under localStorage key cf.

Line 015 - Keep scrolling aligned

```
015 | code.onscroll=()=>{hl.scrollTop=ln.scrollTop=code.scrollTop;hl.scrollLeft=code.scrollLeft};
```

onscroll runs when the textarea scrolls. It copies both the vertical and horizontal scroll positions to hl and ln, so the colored layer and line-number column stay aligned with the textarea. The chained assignments give all elements the same scroll value.

Line 016 - Handle Tab and Run shortcut

```
016 | code.onkeydown=e=>{if(e.key==='Tab'){e.preventDefault();document.execCommand('insertText',fals
    | e,'  ')}else if(e.key==='Enter'&&(e.ctrlKey||e.metaKey)){e.preventDefault();run()}};
```

onkeydown receives a keyboard event e. If the key is Tab, preventDefault cancels normal focus movement and execCommand inserts two spaces. Otherwise, if Enter and either Ctrl or Command/Meta are pressed, it prevents a newline and calls run().

Line 017 - Blank line

No code runs. It separates editor behavior from output formatting.

D. Output formatting

Line 018 - Section comment

```
018 | /* ---------- Output panel ---------- */
```

A visual heading for functions that format and append output.

Line 019 - Format one output value

```
019 | const fmt=v=>typeof v==='string'?v:v instanceof Error?`${v.name}: ${v.message}`:typeof
    | v==='function'?'[fn] '+v.name:v&&typeof v==='object'?(()=>{try{return
    | JSON.stringify(v,null,2)}catch{return String(v)}})():String(v);
```

fmt(v) is a chain of conditional expressions. Strings stay as-is; Error objects become name plus message; functions become a label and function name; objects are pretty-printed as JSON. JSON.stringify is protected by try/catch in case of circular references. Other values use String(v). The question-mark/colon pairs are nested if/else expressions.

Line 020 - Append one output row

```
020 | const log=(t,...a)=>{const d=document.createElement('div');d.className='l
    | '+t;d.textContent=a.map(fmt).join(' ');out.append(d);out.scrollTop=out.scrollHeight};
```

log(t,...a) creates a div. t becomes part of a CSS class such as l log or l error. a collects all values passed after t. Each value is formatted by fmt, joined with spaces, and assigned to textContent. append adds the div to out; the last statement scrolls to the bottom.

Line 021 - Blank line

No code runs. It separates output handling from prompt handling.

E. The custom prompt dialog

Line 022 - Prompt-section comment

```
022 | /* ---------- prompt(): a modal that returns a Promise, so the code waits for your answer
    | ---------- */
```

This comment explains that ask uses a Promise so a program can wait until the user answers.

Line 023 - Create the custom prompt dialog

```
023 | const ask=(q='',d='')=>new Promise(res=>{const m=$('#pm'),i=$('#pi'),done=v=>{m.classList.remo
    | ve('on');res(v)};$('#pq').textContent=q;i.value=d;m.classList.add('on');i.focus();
```

ask(q,d) returns a new Promise. q is the question and d is the initial input, both defaulting to empty text. Inside the Promise executor, m is the modal and i its input. done(v) hides the modal and calls res(v), which resolves the Promise with v. Then the question and initial value are placed into the page, the modal is shown by adding class on, and the input receives focus.

Line 024 - Connect prompt buttons and keys

```
024 |   $('#po').onclick=()=>done(i.value);$('#pc').onclick=()=>done(null);i.onkeydown=e=>{if(e.key=
    | =='Enter')done(i.value);if(e.key==='Escape')done(null)}});
```

Clicking the OK button resolves the Promise with i.value. Clicking Cancel resolves it with null. Enter submits the text; Escape cancels. The Promise remains pending until one of these handlers calls done.

Line 025 - Blank line

No code runs. It separates the prompt helper from the Run function.

F. Compiling and running user code

Line 026 - Run-section comment

```
026 | /* ---------- Run: compile as an async function, so "await prompt()" can pause it ----------
    | */
```

This heading notes that user code is compiled as an async function, which makes await available at its top level.

Line 027 - Find AsyncFunction constructor

```
027 | const AF=Object.getPrototypeOf(async()=>{}).constructor;
```

An async arrow function is created briefly, then its prototype's constructor is read. That constructor can create a new async function from parameter names and source text.

Line 028 - Start the Run function

```
028 | async function run(){
```

Declares an asynchronous function named run. It is called by the Run button and keyboard shortcut. It can use await inside its body.

Line 029 - Reset output and record source/time

```
029 |   out.innerHTML='';stat.className='';stat.textContent='Running...';const
    | src=code.value,t0=performance.now();let err;
```

Clears the old output HTML, clears status styling, shows Running..., copies editor text into src, records the start time, and declares err for a possible error. The comma-separated let declarations are part of one physical line.

Line 030 - Make a custom console

```
030 |   const con={};['log','info','warn','error'].forEach(k=>con[k]=(...a)=>log(k,...a));con.table=
    | con.dir=con.log;
```

con starts as an empty object. forEach visits log, info, warn, and error. For each name k, it creates a method that forwards all arguments to the app's log with k as the output type. table and dir are aliases for the same log function.

Line 031 - Custom async prompt wrapper

```
031 |   const pr=async(q,d)=>{const v=await ask(q,d);log('ask',`[?] ${q}  ->  ${v}`);return v};
```

pr is an async function. It waits for ask(q,d), logs the question and answer, then returns the answer. Because ask resolves after a button or key, this is where the running user program pauses during a custom prompt.

Line 032 - Native prompt fallback

```
032 |   const nat=(q,d)=>{const v=window.prompt(q,d);log('ask',`[?] ${q}  ->  ${v}`);return v};
```

nat calls the browser's built-in window.prompt immediately, logs its result, and returns it. This is used if the rewritten prompt code cannot be compiled.

Line 033 - Begin guarded compilation

```
033 |   try{let f,P=pr;
```

try starts an error-handling region. f will hold the compiled user function. P is initially the custom prompt pr; later it can be switched to nat.

Line 034 - Explain the two prompt paths

```
034 |     // Step 1: turn prompt( into await prompt( . Step 2 (fallback): if that breaks (e.g.
    | prompt inside a normal callback), use the browser's own prompt
```

This comment describes the attempt: rewrite prompt calls to await the custom prompt; if compilation of that rewritten code fails, use the native browser prompt instead.

Line 035 - Compile, or fall back

```
035 |     try{f=new AF('console','prompt','alert',src.replace(/\bprompt\s*\(/g,'await
    | prompt('))}catch{f=new AF('console','prompt','alert',src);P=nat}
```

The inner try creates an async function with parameters named console, prompt, and alert, and uses source text where prompt(was changed to await prompt(. If constructing that function throws, catch creates it from the original source and changes P to nat. This catch surrounds compilation only; errors while the code runs go to the outer catch.

Line 036 - Run the compiled user program

```
036 |     await f(con,P,m=>log('info','[bell] '+m))}
```

await f(...) calls the generated function. The arguments supply the custom console, the chosen prompt function P, and an alert function that logs an info message. Because f is async, run waits for it to finish.

Line 037 - Catch compile or run errors

```
037 |   catch(e){err=e;const m=/<anonymous>:(\d+):\d+/.exec(e.stack||'');log('error',`${e.name}:
    | ${e.message}${m?`  (line ${m[1]-2})`:''}`)}
```

The outer catch receives error e, saves it in err, tries to extract a line number from the stack, and logs the error name/message. The subtraction by 2 compensates for wrapper lines added around the user's code.

Line 038 - Explain empty output

```
038 |   if(!out.children.length)log('muted','No output. Use console.log() to print a result.');
```

If out has no child elements after execution, the app adds a muted reminder to use console.log. A message from an error or prompt would make the panel nonempty.

Line 039 - Set final status

```
039 |   stat.className=err?'bad':'ok';stat.textContent=err?'[X] Error':`[OK] Finished in
    | ${(performance.now()-t0).toFixed(1)} ms`;explain(err)}
```

err ? 'bad' : 'ok' chooses the CSS status class. A second conditional displays either Error or Finished with elapsed milliseconds. toFixed(1) shows one digit after the decimal. Then explain(err) refreshes the teaching panel. This line also closes run.

Line 040 - Listen for unhandled Promise errors

```
040 | addEventListener('unhandledrejection',e=>log('error','Unhandled: '+fmt(e.reason)));
```

The browser fires unhandledrejection if a Promise rejects without a catch. This listener logs its reason. It is separate from run's catch because some later asynchronous work may reject after run has finished.

Line 041 - Blank line

No code runs. It separates execution from the teaching rules.

G. Rules for explaining code

Line 042 - Teaching-rule comment

```
042 | /* ---------- "How it works": each rule = [pattern, concept tag, explanation] ---------- */
```

The comment documents each rule's three parts: matching pattern, topic tag, and explanation function.

Line 043 - Start the rule list

```
043 | const R=[
```

R is an array. Each following entry is an array containing a regular expression, a topic label, and a function that returns an explanation.

Line 044 - Rule: comments

```
044 | [/^\s*\/\/\s*(.*)/,'Comments',m=>`A comment. JavaScript ignores it; it is a note for humans:
    | "${m[1]}".`],
```

Matches a line beginning with optional spaces and //. The captured text is inserted into a sentence explaining that a comment is for humans and ignored by JavaScript.

Line 045 - Rule: prompt input assigned to a variable

```
045 | [/(?:const|let|var)\s+(\w+)\s*=\s*(?:Number\(|parseInt\(|parseFloat\()?\s*(?:await\s+)?prompt\
    | (/,'User input',m=>`Shows a prompt dialog and stores the typed answer in "${m[1]}". prompt()
    | returns text, so numbers must be converted.`],
```

Looks for const/let/var, a variable name, an equals sign, optional number-conversion calls, optional await, and prompt(. It explains that prompt returns text. The capture group supplies the variable name.

Line 046 - Rule: arrow functions

```
046 | [/(?:const|let|var)\s+(\w+)\s*=\s*(?:async\s*)?(?:\([^)]*\)|\w+)\s*=>/,'Functions',m=>`Creates
    | an arrow function "${m[1]}". It runs only when you call it.`],
```

Recognizes a const/let/var assignment whose value resembles an arrow function, either with a parenthesized parameter list or one word parameter. The note says the function runs only when called.

Line 047 - Rule: named functions

```
047 | [/function\s+(\w+)\s*\(([^)]*)\)/,'Functions',m=>`Declares function "${m[1]}"${m[2]?` with
    | inputs (${m[2]})`:''}. Its body runs only when you call ${m[1]}().`],
```

Matches function followed by a name and parameter parentheses. The explanation includes the name and, when present, the parameter text.

Line 048 - Rule: classes

```
048 | [/class\s+(\w+)/,'Classes',m=>`Defines class "${m[1]}", a blueprint for objects made with new
    | ${m[1]}().`],
```

Matches class followed by a name and explains that a class is a blueprint for objects created with new.

Line 049 - Rule: variable declarations

```
049 | [/(const|let|var)\s+(\w+)\s*=/,'Variables',m=>`Declares "${m[2]}" with
    | ${m[1]}${m[1]==='const'?' (cannot be reassigned)':m[1]==='let'?' (can change later)':' (older
    | style, prefer let/const)'} and gives it a value.`],
```

Matches const, let, or var plus a variable name and equals sign. The explanation changes based on which keyword matched.

Line 050 - Rule: for loops

```
050 | [/^\s*for\s*\(/,'Loops',()=>'A for loop: start value, condition, step. The block repeats while
    | the condition is true.'],
```

Recognizes a line beginning with for and describes the initialization, condition, step, and repeated body.

Line 051 - Rule: while loops

```
051 | [/^\s*while\s*\(/,'Loops',()=>'A while loop: repeats the block as long as the condition stays
    | true.'],
```

Recognizes a line beginning with while and explains that its block repeats while the condition stays true.

Line 052 - Rule: if and else-if

```
052 | [/^\s*(?:\}\s*)?else\s+if|^\s*if\s*\(/,'Conditions',()=>'Checks a condition. The block that
    | follows runs only when it is true.'],
```

Recognizes if or else if at the start of a line and explains conditional execution.

Line 053 - Rule: else

```
053 | [/^\s*(?:\}\s*)?else\b/,'Conditions',()=>'Runs when none of the earlier conditions were
    | true.'],
```

Recognizes else and explains that it runs when earlier conditions were false.

Line 054 - Rule: switch

```
054 | [/^\s*switch\s*\(/,'Conditions',()=>'Compares one value against several case labels and runs
    | the matching one.'],
```

Recognizes switch and explains that it chooses among cases by comparing one value.

Line 055 - Rule: array methods

```
055 | [/\.(map|filter|reduce|forEach|find|some|every)\(/,'Array methods',m=>`Calls .${m[1]}(), which
    | runs a callback on each array item${{map:' and builds a new array of the results',filter:' and
    | keeps only the items that pass the test',reduce:' and combines them into one value',forEach:'
    | (used for side effects like printing)'}[m[1]]||''}.`],
```

Looks for common array methods after a dot. It inserts the matched method name into the explanation. map creates a transformed array, filter keeps matching elements, reduce combines values, and forEach is often used for effects such as printing.

Line 056 - Rule: console output

```
056 | [/console\.(log|info|warn|error)\(/,'Output',m=>`Prints a ${m[1]==='error'?'red
    | error':m[1]==='warn'?'warning':'message'} to the Output panel.`],
```

Matches console.log/info/warn/error and selects an explanation based on which method was found.

Line 057 - Rule: alert

```
057 | [/\balert\(/,'Output',()=>'Shows a message in the Output panel (alert is redirected here).'],
```

Matches alert(and explains that this app redirects alerts into its output panel.

Line 058 - Rule: prompt

```
058 | [/\bprompt\(/,'User input',()=>'Asks the user to type a value; the answer comes back as
    | text.'],
```

Matches prompt(and explains that it asks for text.

Line 059 - Rule: asynchronous concepts

```
059 | [/\b(?:await|async|setTimeout|setInterval)\b|\.then\(/,'Async',()=>'Asynchronous code: it runs
    | later or waits for a Promise, without freezing the page.'],
```

Matches the words await, async, setTimeout, setInterval, or the text .then(. It labels the line Async. This rule only recognizes text; it does not itself run timers.

Line 060 - Rule: error handling

```
060 | [/^\s*(?:\}\s*)?try\b|catch\s*\(|\bthrow\b/,'Error handling',()=>'try runs risky code, catch
    | handles any error it throws, throw raises one.'],
```

Recognizes try, catch, or throw and gives a short explanation of error handling.

Line 061 - Rule: return and close the array

```
061 | [/^\s*return\b/,'Functions',()=>'Sends a value back to the caller and ends the function.']];
```

Recognizes a return line and explains it. The final]; closes the R array.

Line 062 - Error explanations

```
062 | const HINT={SyntaxError:'The code is not valid JavaScript. Check for a missing bracket, quote,
    | comma or brace near the last place you edited.',ReferenceError:'A name is used that does not
    | exist. Check the spelling, or declare it with let/const before using it.',TypeError:'A value
    | was used in a way its type does not allow, such as calling something that is not a function or
    | reading a property of undefined.',RangeError:'A number or size is outside the allowed range,
    | for example infinite recursion.'};
```

HINT is an object whose property names are error classes. For example, HINT.SyntaxError returns the syntax hint. The fallback later handles error names not listed here.

H. Building the explanation panel

Line 063 - Begin explain function

```
063 | function explain(err){
```

Declares explain(err). err is either the error from the latest run or no error.

Line 064 - Prepare explanations and unique topics

```
064 |   const rows=[],tags=new Set();
```

rows collects rendered explanation rows. tags is a Set so repeated topic labels appear only once.

Line 065 - Scan each source line

```
065 |   code.value.split('\n').forEach(l=>{if(/^[\s{}()\];,]*$/.test(l))return;const r=R.find(r=>r[0
    | ].test(l));if(r){tags.add(r[1]);rows.push(`<li><code>${esc(l.trim())}</code><span>${r[2](l.mat
    | ch(r[0]))}</span></li>`)}});
```

Splits code into lines and visits each line l. A regular expression skips blank or punctuation-only lines. R.find picks the first matching rule. If found, its topic is added and an li is built with escaped source text and the rule's explanation. The source is escaped so typed code is displayed as text.

Line 066 - Start building the explanation panel

```
066 |   why.innerHTML=(err?`<div class="hint"><b>${esc(err.name)}</b>: ${HINT[err.name]||'The code
    | threw an error while running. Read the message in the Output panel.'}</div>`:'')+
```

Assigns the final HTML to why.innerHTML. If err exists, it first creates an error hint. HINT[err.name] uses a known explanation, or the text after || is used as a default.

Line 067 - Add topics and explanation list

```
067 |     (rows.length?`<div
    | class="chips">${[...tags].map(t=>`<em>${t}</em>`).join('')}</div><ol>${rows.join('')}</ol><p
    | class="note">JavaScript runs top to bottom, one statement at a time. Functions wait until they
    | are called.</p>`:'<p class="note">Type some code and press Run.</p>')}
```

If rows exist, the code makes topic chips and an ordered list from rows, plus a general note. If no source lines matched any rule, it displays a prompt to type code. This closes explain.

Line 068 - Blank line

No code runs. It separates explanation logic from event wiring.

I. Buttons, browser events, and startup

Line 069 - Wiring comment

```
069 | /* ---------- Wiring ---------- */
```

A heading for the code that connects page controls and browser events to functions.

Line 070 - Run button

```
070 | $('#run').onclick=run;
```

Assigns run as the click handler for the element with ID run.

Line 071 - Clear button

```
071 | $('#clear').onclick=()=>{code.value='';sync();out.innerHTML='';stat.textContent='';explain()};
```

Clicking Clear empties the editor, refreshes highlight/line numbers, clears output and visible status text, and rebuilds explanations. It does not call save, so a prior saved draft may return after a page reload.

Line 072 - Theme selection

```
072 | $('#theme').onchange=e=>{document.documentElement.dataset.theme=e.target.value;save('cft',e.ta
    | rget.value)};
```

When the theme selector changes, the chosen value is written to document.documentElement.dataset.theme for CSS to read and saved as cft.

Line 073 - Restore theme

```
073 | const th=load('cft');if(th){$('#theme').value=th;document.documentElement.dataset.theme=th}
```

Loads cft from localStorage. If a saved theme exists, it selects it in the dropdown and applies it to the HTML root.

Line 074 - Close the drawer

```
074 | $('.drawer').onclick=()=>$('#menu').checked=false;
```

Clicking an element with class drawer sets the checkbox menu to false. This likely closes a CSS-controlled navigation drawer.

Line 075 - Style page after scrolling

```
075 | addEventListener('scroll',()=>document.body.classList.toggle('sc',scrollY>30));
```

A page scroll listener toggles class sc on body depending on whether scrollY exceeds 30 pixels. toggle(class, condition) adds the class when true and removes it when false.

Line 076 - Track pointer position over cards

```
076 | document.addEventListener('pointermove',e=>{const c=e.target.closest('.card');if(c){const k=c.
    | getBoundingClientRect();c.style.setProperty('--x',e.clientX-k.left+'px');c.style.setProperty('
    | --y',e.clientY-k.top+'px')}});
```

On pointer movement, closest('.card') finds the nearest card containing the target. If one exists, getBoundingClientRect gives its position; the pointer's page coordinates are converted to card-relative x/y and saved as CSS custom properties --x

and --y.

Line 077 - Make a click ripple and remove it later

```
077 | document.addEventListener('click',e=>{const b=e.target.closest('.btn');if(!b)return;const r=do
    | cument.createElement('i'),k=b.getBoundingClientRect();r.className='rip';r.style.cssText=`left:
    | ${e.clientX-k.left}px;top:${e.clientY-k.top}px`;b.append(r);setTimeout(()=>r.remove(),600)});
```

On any click, closest('.btn') finds a button ancestor. If none exists, return exits the handler. Otherwise it creates an i element, computes click position inside the button, assigns class rip and inline coordinates, and appends it. setTimeout schedules r.remove() after 600 milliseconds so the temporary ripple disappears.

Line 078 - Restore editor text and initialize views

```
078 | code.value=load('cf')||code.value;sync();explain();
```

Loads saved code cf, or keeps the current HTML-provided starter text if there is no saved value. Then it syncs highlighting/line numbers and generates teaching explanations. This is the startup sequence.

4. The full Run journey, traced with examples

Here is what happens when the learner clicks Run, in order:

- The click handler calls run().
- run clears the output, marks the page as Running, copies the editor text, and starts a timer for measuring elapsed time.
- run creates replacement console and prompt functions, then tries to turn the source text into an async function.
- The generated function receives the replacement console, prompt, and alert as its parameters.
- User code runs. Calls to console.log become rows in the page output. A custom prompt pauses while waiting for a button or key.
- After completion, run displays either a success status and elapsed time or an error message.
- Finally explain scans the source text and adds any matching teaching notes.

Example A: console output

If the learner enters console.log('Hello'), the generated function calls the custom console.log. That method calls log('log', 'Hello'). log creates a div, puts Hello into its text, appends it to out, and scrolls to the bottom. When execution ends, run sets the Finished status.

Example B: ask for a name

For a top-level prompt call, the rewriter adds await before prompt(. The prompt parameter refers to pr. pr calls ask, which opens the modal and returns a Promise. The user's click runs done(answer), which resolves that Promise. await then resumes the user's code with the answer. The answer is text, even if the person types digits.

Example C: an error

If the user's code refers to a variable that was never declared, JavaScript may throw ReferenceError. The outer catch receives the Error object, log displays its name and message, run marks status bad, and explain looks up a beginner hint by the error's name.

Important detail about the prompt fallback

The inner catch surrounds creation of the generated function. It is intended for syntax errors caused by adding await in a place where await is not legal. If that compilation attempt fails, it tries the original source and switches to the browser's synchronous prompt. Errors thrown after a generated function begins running are handled by the outer catch instead.

5. async functions, Promises, and await

These ideas are the key to understanding why the custom prompt works without freezing the page.

What is asynchronous work?

Synchronous work happens step by step and the current code waits for each step to finish. Asynchronous work is arranged to finish later. While it is waiting, the browser can still respond to clicks, typing, and drawing the page.

What is a Promise?

A Promise is an object representing a result that may arrive later. Its state starts pending and eventually becomes fulfilled with a value or rejected with an error. ask returns a Promise. It is pending while the dialog is visible and becomes fulfilled when done calls res(value).

What does async mean?

Putting async before a function means that the function always returns a Promise. Inside it, await is allowed. Even if the function returns an ordinary value, JavaScript wraps the result in a fulfilled Promise.

What does await mean?

await tells the current async function to pause at that point until the Promise settles. When the Promise fulfills, the expression becomes its result and the function continues. It does not freeze the whole browser page.

The prompt connection

- The learner's code calls prompt(question).
- The source rewrite makes the call await prompt(question).
- The prompt parameter is pr, which awaits ask(question).
- ask shows the dialog and waits. The Promise is still pending, so pr and then the learner code are paused.
- The learner clicks OK. The click handler calls done(inputValue). done calls res(inputValue), fulfilling the Promise.
- The await in ask's caller receives the text. Execution continues after the prompt line.

Small Promise example

```
const wait = ms => new Promise(resolve => setTimeout(resolve, ms));
async function demo() {
  console.log('before');
  await wait(500);
  console.log('after about half a second');
}
demo();
```

Read it as: wait creates a Promise that will resolve after a timer. demo starts, prints before, pauses at await, and later prints after. The browser can do other work during the pause.

async does not mean 'run on another thread'

Async/await organizes waiting; it does not automatically move CPU-heavy calculations to another processor. A long loop can still make a page unresponsive. This script's custom prompt is a wait for user interaction, so async/await is a natural fit.

6. setTimeout and the click ripple

The only actual timer call in the pasted program is on line 77: setTimeout(() => r.remove(), 600). The rules table mentions setTimeout and setInterval because the teaching panel wants to recognize those words in learner code. That rule does not itself start either timer.

How the line works

- setTimeout takes a callback function and a delay measured in milliseconds.
- The callback here is () => r.remove(). It means: when called, remove the ripple element r from the page.
- 600 milliseconds is six tenths of a second.
- The browser schedules the callback and immediately continues with other work. It does not pause the click handler for 600 milliseconds.
- After at least about 600 milliseconds, when the browser can run queued work, the callback removes the temporary element.

Difference from setInterval

setTimeout schedules one later call. setInterval schedules repeated calls at an interval until cancelled. There is no setInterval call in this playground's own behavior; setInterval appears only as text in the Async explanation rule.

Timeline

```
click happens -> create ripple -> schedule removal in 600 ms -> handler finishes
other browser work continues -> timer callback runs -> ripple is removed
```

7. Regular expressions and the teaching-rule engine

A regular expression, often called a regex, is a search pattern. This code uses regexes twice: first to identify pieces for syntax coloring, and later to identify source lines for short explanations.

Regex piece	Meaning
`/pattern/g`	A regex literal. The g flag means keep finding matches throughout the text.
`^ and $`	Beginning and end of a line/string. Used to recognize a line that starts with a comment or loop.
`\s`	Whitespace such as spaces or tabs. \s* means zero or more whitespace characters.
`\d`	A digit. \d+ means one or more digits.
`\w`	A common word character: letter, digit, or underscore.
`* + ?`	Repeat counts: * means zero or more, + one or more, and ? makes a piece optional or makes a repeat non-greedy after *.
`(...)`	Groups a pattern. A capture group can later be read from match[1], match[2], etc.
`(?:...)`	Groups a pattern without capturing a result number.
`[...]`	Matches one character from a set or range. [A-Za-z_$] means a letter, underscore, or dollar sign.
`(?=...)`	A lookahead: checks what comes next without consuming it. Here it recognizes a name before an opening parenthesis.
`|`	OR between alternatives.
`\b`	A word boundary, used to avoid matching a name only as part of a longer word.

How explain chooses a note

For each line, explain tries R.find. find checks rules in order and stops at the first matching rule. The rule's regular expression recognizes the line, the tag adds a topic chip, and the third item is called to produce the sentence. Because it stops at the first match, a line that could fit multiple patterns receives only the earliest rule's explanation.

Why the explanation code uses escape

A learner's source is untrusted text. The explanation list places that source inside HTML, so esc changes special characters before insertion. The description itself is generated by this program's own rule functions. log uses textContent for output, which also treats output as text rather than HTML.

8. Important names and browser methods

Name	Role
`$`	Shortcut for document.querySelector.
`code.value`	The text currently in the editor.
`hl / ln`	Highlighted code layer / line-number layer.
`out / why / stat`	Output panel / explanation panel / run-status element.
`save / load`	Small wrappers around localStorage.
`KW / BI`	Sets used to classify keywords and built-in names for coloring.
`TOK / hi`	Token regex / function that converts source into highlighted HTML.
`sync`	Refreshes highlighting and line numbers from code.value.
`fmt / log`	Format values / append formatted output.
`ask / pr / nat`	Custom modal prompt / async wrapper for it / native browser prompt.
`AF / f`	AsyncFunction constructor / generated function made from learner source.
`con / P`	Custom console object / whichever prompt function is currently selected.
`src / t0 / err`	Source snapshot / start time / caught error.
`R / HINT / explain`	Teaching rules / error messages / function that builds the explanation panel.
`th`	Previously saved theme value.
`e`	An event object supplied by the browser to a handler.
`m, i, v, q, d`	Depending on function: modal, input, resolved value, question, initial/default text.
`t, a, k`	Output type / extra arguments array / current method name in forEach.
`c, r`	In different handlers: card element or CSS class, and result/ripple element.

Browser methods worth recognizing

- document.querySelector(selector) finds the first matching element.
- addEventListener(eventName, callback) asks the browser to call a function when an event occurs.
- element.append(child) adds a child element.
- element.classList.add/remove/toggle changes CSS classes.
- element.textContent sets visible text; element.innerHTML reads or writes HTML markup.
- element.getBoundingClientRect() gives the element's position and size on screen.
- element.closest(selector) searches the element and its ancestors for a match.
- localStorage stores small strings in the browser across page reloads.
- performance.now() gives a precise time reading useful for measuring duration.

9. What the program does not do, plus careful-reading notes

A few details help avoid misunderstandings while learning:

- The file is only the JavaScript behavior. The HTML and CSS that create the editor, panels, modal, themes, cards, and ripple appearance are not included in this pasted text.
- The highlighter is a regex-based colorizer. It can color common syntax, but it does not fully parse JavaScript grammar. Complex nested expressions can be colored imperfectly without affecting actual execution.
- The prompt rewrite operates on raw source characters. It does not know whether prompt(appears in executable code, a comment, or a quoted string. This is a useful design limitation to understand when debugging the playground.
- The generated function receives custom console, prompt, and alert parameters, but it still runs in the page's JavaScript environment and can access browser globals such as document and window. This is not an isolated secure sandbox.
- setTimeout's delay is a minimum scheduling delay, not a promise that the callback runs at exactly 600.000 milliseconds. Busy browser work can make it run later.
- The Clear handler clears what is on screen but does not save the empty string. If a saved draft exists, it can be restored after reloading.
- The program's teaching rules are heuristic: they search lines of text. They do not prove the user's code is correct or understand every possible syntax form.

One small manual trace

```
Start page
  load saved code -> sync() -> explain()

Type a character
  oninput -> sync() -> save('cf', code.value)

Click Run
  run() -> build async function -> execute -> log output
        -> set status -> explain(err)

Click inside a button
  click listener -> add ripple -> setTimeout -> remove ripple
```

Appendix. Complete pasted source, numbered

This is the complete 78-line source as supplied. Long physical lines wrap visually but keep their original line number. A few display-only symbols are written as [bell], [OK], [X], [?], or [fn] so they remain readable in a standard PDF font; those stand for the original icon characters.

```
001 | const $=s=>document.querySelector(s),code=$('#code'),hl=$('#hl'),ln=$('#ln'),out=$('#out'),
    | why=$('#why'),stat=$('#stat');
002 | const save=(k,v)=>{try{localStorage[k]=v}catch{}},load=k=>{try{return
    | localStorage[k]}catch{}};
003 |
004 | /* ---------- Syntax highlighting: split code into tokens, wrap each in a coloured span
    | ---------- */
005 | const KW=new Set('const let var function return if else for while do switch case break
    | continue new class extends try catch finally throw async await of in typeof instanceof this
    | null undefined true false default yield delete void'.split(' '));
006 | const BI=new Set('console Math JSON Object Array Number String Boolean Promise Date Map Set
    | document window'.split(' '));
007 | const esc=s=>s.replace(/&/g,'&amp;').replace(/</g,'&lt;');
008 | // groups: 1 comment, 2 string, 3 number, 4 name before "(", 5 other name, 6 operator
009 | const TOK=/(\/\/.*|\/\*[\s\S]*?\*\/)|("(?:\\.|[^"\\\n])*"|'(?:\\.|[^'\\\n])*'|`(?:\\.|[^`\\
    | ])*`)|(\b\d+\.?\d*\b)|([A-Za-z_$][\w$]*)(?=\s*\()|([A-Za-z_$][\w$]*)|([=+\-*/%<>!&|?:]+)/g;
010 | const hi=s=>{let r='',i=0,m;TOK.lastIndex=0;while(m=TOK.exec(s)){const t=m[0],c=m[1]?'com':
    | m[2]?'str':m[3]?'num':m[4]||m[5]?(KW.has(t)?'kw':BI.has(t)?'bi':m[4]?'fn':''):'op';r+=esc(s
    | .slice(i,m.index))+(c?`<span class="${c}">${esc(t)}</span>`:esc(t));i=TOK.lastIndex}return
    | r+esc(s.slice(i))};
011 |
012 | // The coloured <pre> sits under a transparent <textarea>; keep text, line numbers and
    | scroll in sync
013 | const sync=()=>{hl.innerHTML=hi(code.value)+'\n
    | ';ln.textContent=code.value.split('\n').map((_,i)=>i+1).join('\n')+'\n ';};
014 | code.oninput=()=>{sync();save('cf',code.value)};
015 | code.onscroll=()=>{hl.scrollTop=ln.scrollTop=code.scrollTop;hl.scrollLeft=code.scrollLeft};
016 | code.onkeydown=e=>{if(e.key==='Tab'){e.preventDefault();document.execCommand('insertText',f
    | alse,'  ')}else if(e.key==='Enter'&&(e.ctrlKey||e.metaKey)){e.preventDefault();run()}};
017 |
018 | /* ---------- Output panel ---------- */
019 | const fmt=v=>typeof v==='string'?v:v instanceof Error?`${v.name}: ${v.message}`:typeof
    | v==='function'?'[fn] '+v.name:v&&typeof v==='object'?(()=>{try{return
    | JSON.stringify(v,null,2)}catch{return String(v)}})():String(v);
020 | const log=(t,...a)=>{const d=document.createElement('div');d.className='l
    | '+t;d.textContent=a.map(fmt).join(' ');out.append(d);out.scrollTop=out.scrollHeight};
021 |
022 | /* ---------- prompt(): a modal that returns a Promise, so the code waits for your answer
    | ---------- */
023 | const ask=(q='',d='')=>new Promise(res=>{const m=$('#pm'),i=$('#pi'),done=v=>{m.classList.r
    | emove('on');res(v)};$('#pq').textContent=q;i.value=d;m.classList.add('on');i.focus();
024 |   $('#po').onclick=()=>done(i.value);$('#pc').onclick=()=>done(null);i.onkeydown=e=>{if(e.k
    | ey==='Enter')done(i.value);if(e.key==='Escape')done(null)}});
025 |
026 | /* ---------- Run: compile as an async function, so "await prompt()" can pause it
    | ---------- */
027 | const AF=Object.getPrototypeOf(async()=>{}).constructor;
028 | async function run(){
029 |   out.innerHTML='';stat.className='';stat.textContent='Running...';const
    | src=code.value,t0=performance.now();let err;
030 |   const con={};['log','info','warn','error'].forEach(k=>con[k]=(...a)=>log(k,...a));con.tab
    | le=con.dir=con.log;
031 |   const pr=async(q,d)=>{const v=await ask(q,d);log('ask',`[?] ${q}  ->  ${v}`);return v};
```

```
032 |   const nat=(q,d)=>{const v=window.prompt(q,d);log('ask',`[?] ${q}  ->  ${v}`);return v};
033 |   try{let f,P=pr;
034 |     // Step 1: turn prompt( into await prompt( . Step 2 (fallback): if that breaks (e.g.
    | prompt inside a normal callback), use the browser's own prompt
035 |     try{f=new AF('console','prompt','alert',src.replace(/\bprompt\s*\(/g,'await
    | prompt('))}catch{f=new AF('console','prompt','alert',src);P=nat}
036 |     await f(con,P,m=>log('info','[bell] '+m))}
037 |   catch(e){err=e;const m=/<anonymous>:(\d+):\d+/.exec(e.stack||'');log('error',`${e.name}:
    | ${e.message}${m?`  (line ${m[1]-2})`:''}`)}
038 |   if(!out.children.length)log('muted','No output. Use console.log() to print a result.');
039 |   stat.className=err?'bad':'ok';stat.textContent=err?'[X] Error':`[OK] Finished in
    | ${(performance.now()-t0).toFixed(1)} ms`;explain(err)}
040 | addEventListener('unhandledrejection',e=>log('error','Unhandled: '+fmt(e.reason)));
041 |
042 | /* ---------- "How it works": each rule = [pattern, concept tag, explanation] ---------- */
043 | const R=[
044 | [/^\s*\/\/\s*(.*)/,'Comments',m=>`A comment. JavaScript ignores it; it is a note for
    | humans: "${m[1]}".`],
045 | [/(?:const|let|var)\s+(\w+)\s*=\s*(?:Number\(|parseInt\(|parseFloat\()?\s*(?:await\s+)?prom
    | pt\(/,'User input',m=>`Shows a prompt dialog and stores the typed answer in "${m[1]}".
    | prompt() returns text, so numbers must be converted.`],
046 | [/(?:const|let|var)\s+(\w+)\s*=\s*(?:async\s*)?(?:\([^)]*\)|\w+)\s*=>/,'Functions',m=>`Crea
    | tes an arrow function "${m[1]}". It runs only when you call it.`],
047 | [/function\s+(\w+)\s*\(([^)]*)\)/,'Functions',m=>`Declares function "${m[1]}"${m[2]?` with
    | inputs (${m[2]})`:''}. Its body runs only when you call ${m[1]}().`],
048 | [/class\s+(\w+)/,'Classes',m=>`Defines class "${m[1]}", a blueprint for objects made with
    | new ${m[1]}().`],
049 | [/(const|let|var)\s+(\w+)\s*=/,'Variables',m=>`Declares "${m[2]}" with
    | ${m[1]}${m[1]==='const'?' (cannot be reassigned)':m[1]==='let'?' (can change later)':'
    | (older style, prefer let/const)'} and gives it a value.`],
050 | [/^\s*for\s*\(/,'Loops',()=>'A for loop: start value, condition, step. The block repeats
    | while the condition is true.'],
051 | [/^\s*while\s*\(/,'Loops',()=>'A while loop: repeats the block as long as the condition
    | stays true.'],
052 | [/^\s*(?:\}\s*)?else\s+if|^\s*if\s*\(/,'Conditions',()=>'Checks a condition. The block that
    | follows runs only when it is true.'],
053 | [/^\s*(?:\}\s*)?else\b/,'Conditions',()=>'Runs when none of the earlier conditions were
    | true.'],
054 | [/^\s*switch\s*\(/,'Conditions',()=>'Compares one value against several case labels and
    | runs the matching one.'],
055 | [/\.(map|filter|reduce|forEach|find|some|every)\(/,'Array methods',m=>`Calls .${m[1]}(),
    | which runs a callback on each array item${{map:' and builds a new array of the
    | results',filter:' and keeps only the items that pass the test',reduce:' and combines them
    | into one value',forEach:' (used for side effects like printing)'}[m[1]]||''}.`],
056 | [/console\.(log|info|warn|error)\(/,'Output',m=>`Prints a ${m[1]==='error'?'red
    | error':m[1]==='warn'?'warning':'message'} to the Output panel.`],
057 | [/\balert\(/,'Output',()=>'Shows a message in the Output panel (alert is redirected
    | here).'],
058 | [/\bprompt\(/,'User input',()=>'Asks the user to type a value; the answer comes back as
    | text.'],
059 | [/\b(?:await|async|setTimeout|setInterval)\b|\.then\(/,'Async',()=>'Asynchronous code: it
    | runs later or waits for a Promise, without freezing the page.'],
060 | [/^\s*(?:\}\s*)?try\b|catch\s*\(|\bthrow\b/,'Error handling',()=>'try runs risky code,
    | catch handles any error it throws, throw raises one.'],
061 | [/^\s*return\b/,'Functions',()=>'Sends a value back to the caller and ends the
    | function.']];
062 | const HINT={SyntaxError:'The code is not valid JavaScript. Check for a missing bracket,
    | quote, comma or brace near the last place you edited.',ReferenceError:'A name is used that
    | does not exist. Check the spelling, or declare it with let/const before using
    | it.',TypeError:'A value was used in a way its type does not allow, such as calling
    | something that is not a function or reading a property of undefined.',RangeError:'A number
```

```
    | or size is outside the allowed range, for example infinite recursion.'};
063 | function explain(err){
064 |   const rows=[],tags=new Set();
065 |   code.value.split('\n').forEach(l=>{if(/^[\s{}()\];,]*$/.test(l))return;const r=R.find(r=>
    | r[0].test(l));if(r){tags.add(r[1]);rows.push(`<li><code>${esc(l.trim())}</code><span>${r[2]
    | (l.match(r[0]))}</span></li>`)}});
066 |   why.innerHTML=(err?`<div class="hint"><b>${esc(err.name)}</b>: ${HINT[err.name]||'The
    | code threw an error while running. Read the message in the Output panel.'}</div>`:'')+
067 |     (rows.length?`<div class="chips">${[...tags].map(t=>`<em>${t}</em>`).join('')}</div><ol
    | >${rows.join('')}</ol><p class="note">JavaScript runs top to bottom, one statement at a
    | time. Functions wait until they are called.</p>`:'<p class="note">Type some code and press
    | Run.</p>')}
068 |
069 | /* ---------- Wiring ---------- */
070 | $('#run').onclick=run;
071 | $('#clear').onclick=()=>{code.value='';sync();out.innerHTML='';stat.textContent='';explain(
    | )};
072 | $('#theme').onchange=e=>{document.documentElement.dataset.theme=e.target.value;save('cft',e
    | .target.value)};
073 | const th=load('cft');if(th){$('#theme').value=th;document.documentElement.dataset.theme=th}
074 | $('.drawer').onclick=()=>$('#menu').checked=false;
075 | addEventListener('scroll',()=>document.body.classList.toggle('sc',scrollY>30));
076 | document.addEventListener('pointermove',e=>{const c=e.target.closest('.card');if(c){const k
    | =c.getBoundingClientRect();c.style.setProperty('--x',e.clientX-k.left+'px');c.style.setProp
    | erty('--y',e.clientY-k.top+'px')}});
077 | document.addEventListener('click',e=>{const b=e.target.closest('.btn');if(!b)return;const r
    | =document.createElement('i'),k=b.getBoundingClientRect();r.className='rip';r.style.cssText=
    | `left:${e.clientX-k.left}px;top:${e.clientY-k.top}px`;b.append(r);setTimeout(()=>r.remove()
    | ,600)});
078 | code.value=load('cf')||code.value;sync();explain();
```